CS343 Operating Systems

Lecture 16

Virtual Memory Techniques

John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Structure of the Page Table

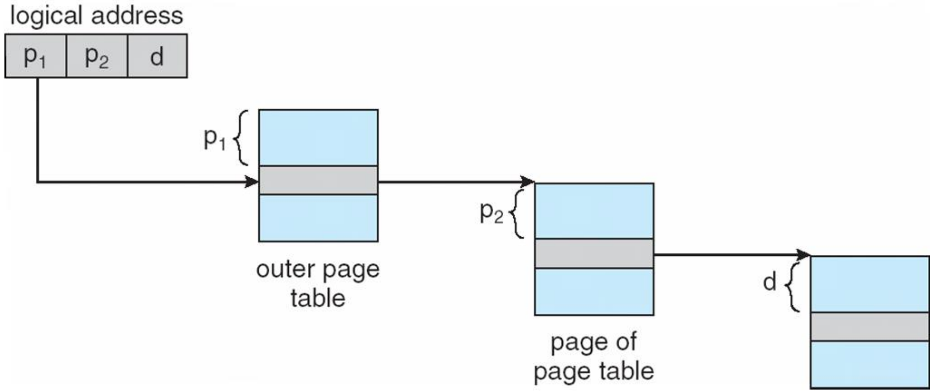
- Memory structures for paging can get huge using straight-forward methods
  - Hierarchical Paging
  - Hashed Page Tables
  - Inverted Page Tables

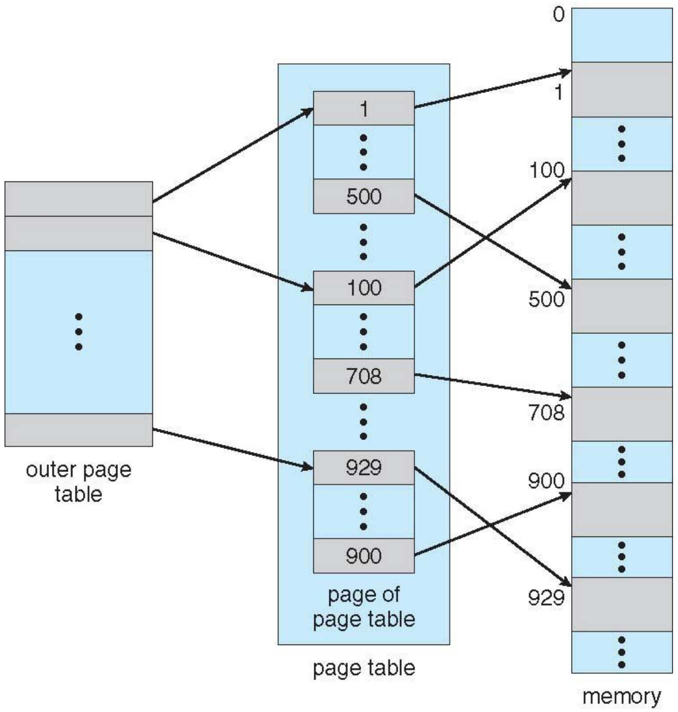
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table

Address-Translation Scheme

page number		page offset
p_1	p_2	d
12	10	10

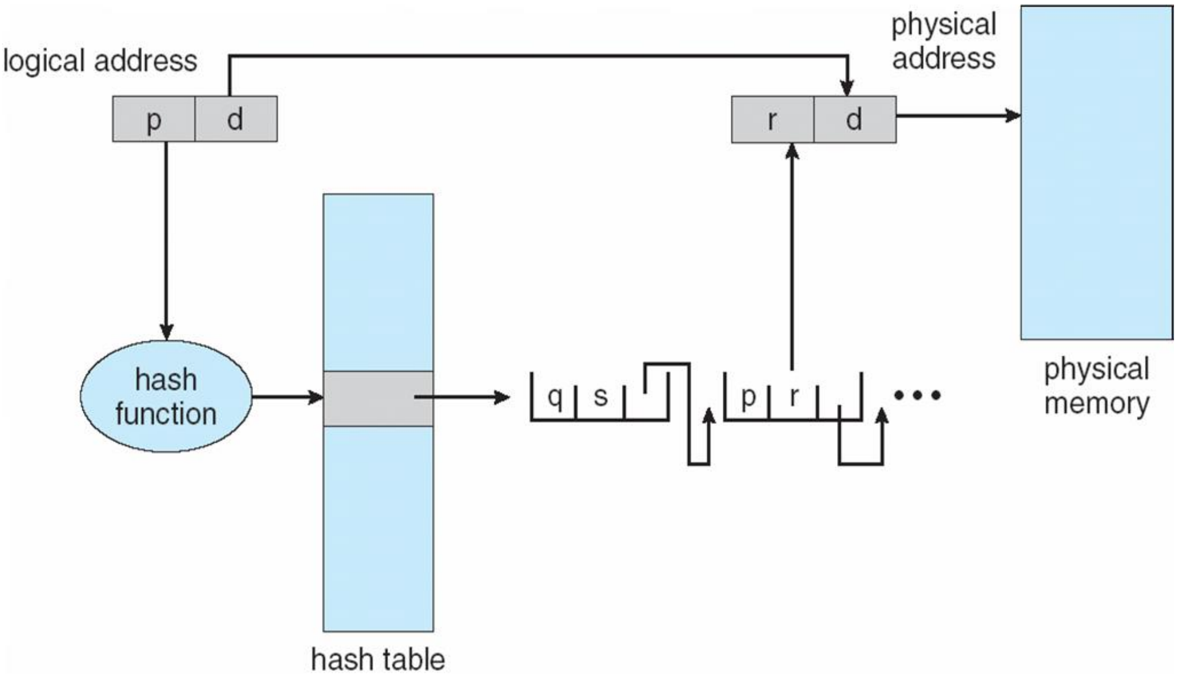




Hashed Page Tables

- The virtual page number is hashed into a page table
- This page table contains a chain of elements hashing to the same location
- Each element contains
  - (1) the virtual page number
  - (2) the value of the mapped page frame
  - (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
- If a match is found, the corresponding physical frame is extracted

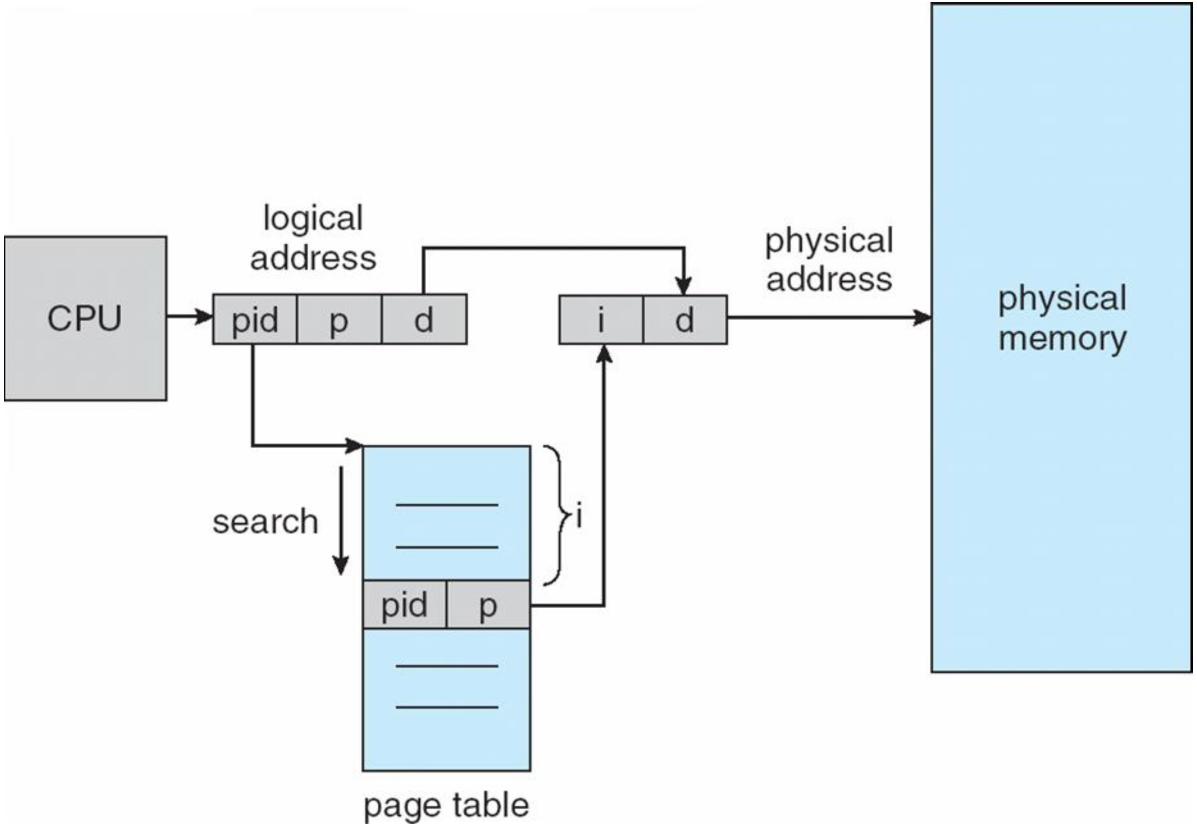
Hashed Page Tables



Inverted Page Table

- No page table per process
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table,
- Increases time needed to search the table when a page reference occurs

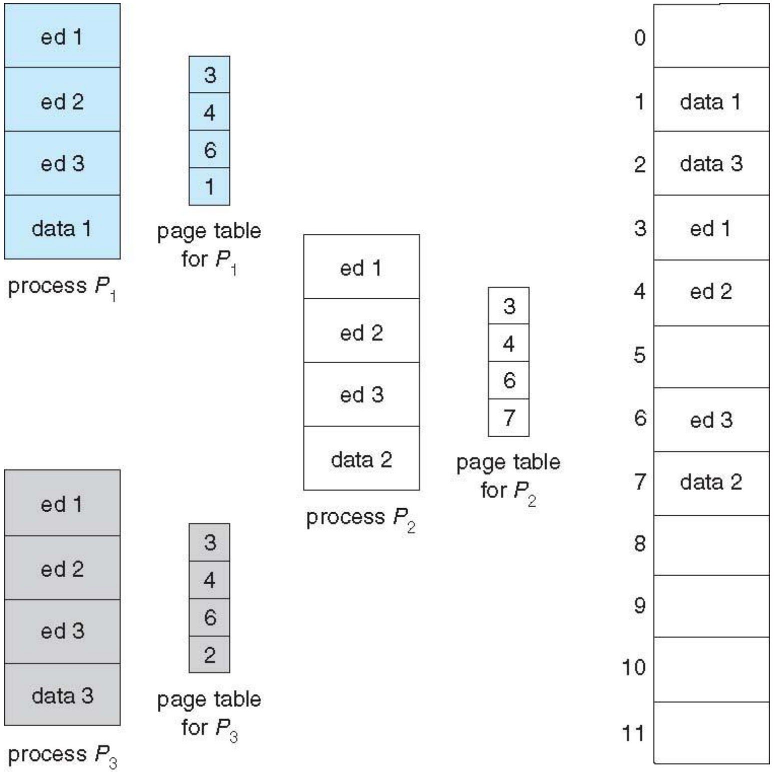
Inverted Page Table Architecture



Shared Pages

- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
  - Similar to multiple threads sharing the same process space
  - Also useful for interprocess communication if sharing of read-write pages is allowed
- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages



Background

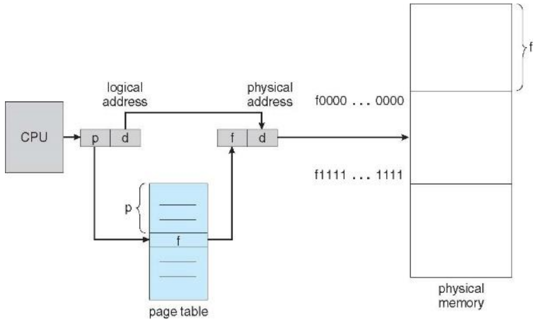
- Code needs to be in memory to execute
- But entire program code not needed at same time
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running → more programs run at the same time
  - Increased CPU utilization and throughput with no increase in response time or turnaround time

Background

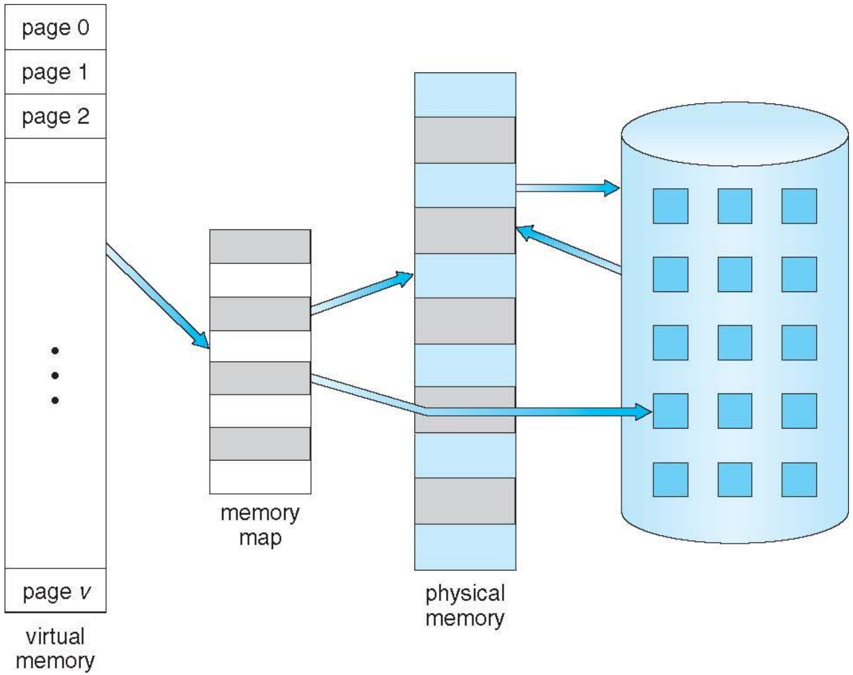
- **Virtual memory** – separation of user logical memory from physical memory
- Only part of the program needs to be in memory for execution
- Logical address space can therefore be much larger than physical address space
- Allows address spaces to be shared by several processes
- More programs running concurrently
- Less I/O needed to load or swap processes

Background

- **Virtual address space** – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via Demand paging

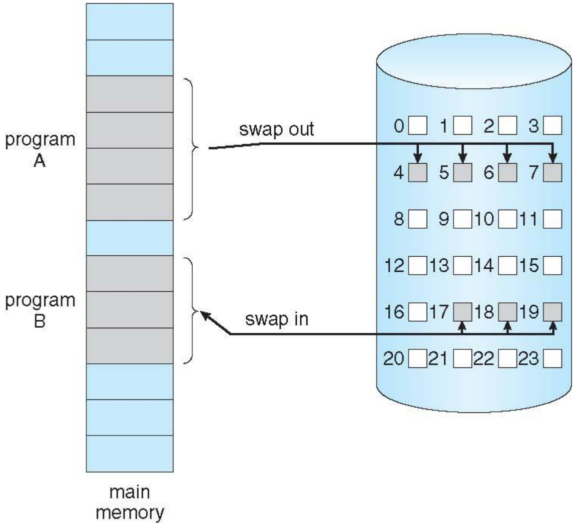


Virtual Memory To Physical Memory Mapping



Demand Paging

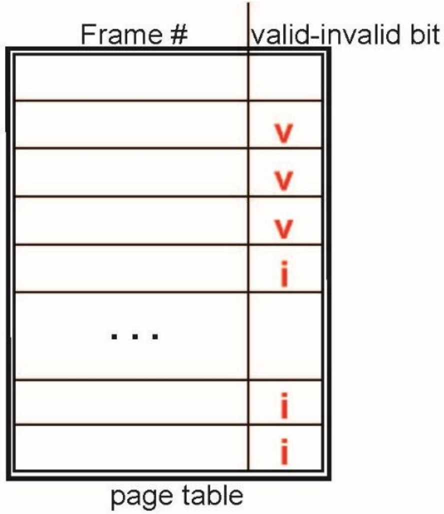
- Bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- **Lazy swapper** – never swaps a page into memory unless page will be needed

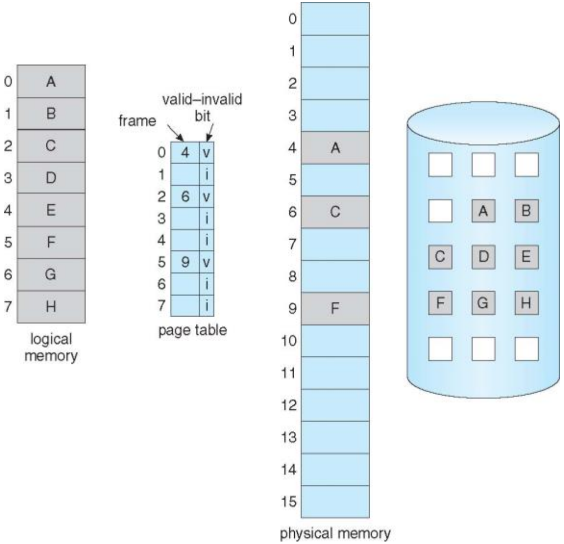


Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated
  (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)

- Initially valid–invalid bit is set to **i** on all entries



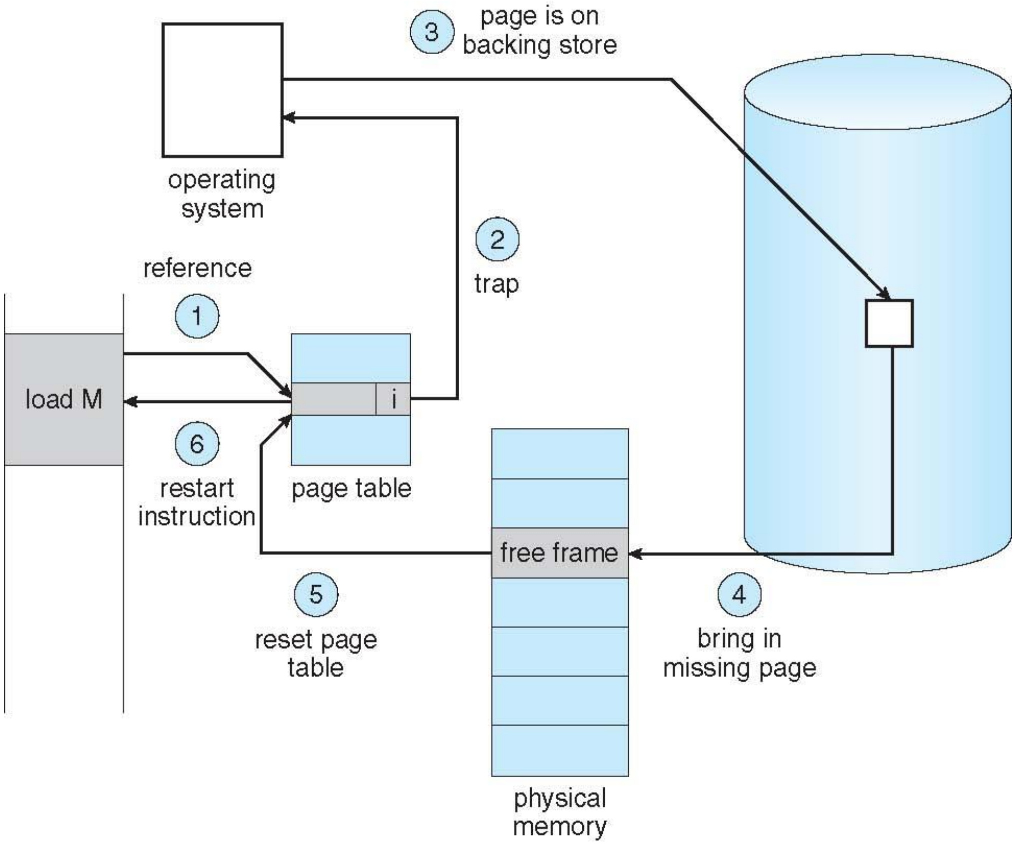


Page Table When Some Pages Are Not in Main Memory

Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system: **page fault** (page not found in main memory)
- Find free frame
- Swap page into frame via scheduled disk operation
- Reset tables to indicate page now in memory: Set validation bit = **1**
- Restart the instruction that caused the page fault

Steps in Handling a Page Fault



Demand Paging Overhead

- Page Fault Rate $0 \le p \le 1$
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

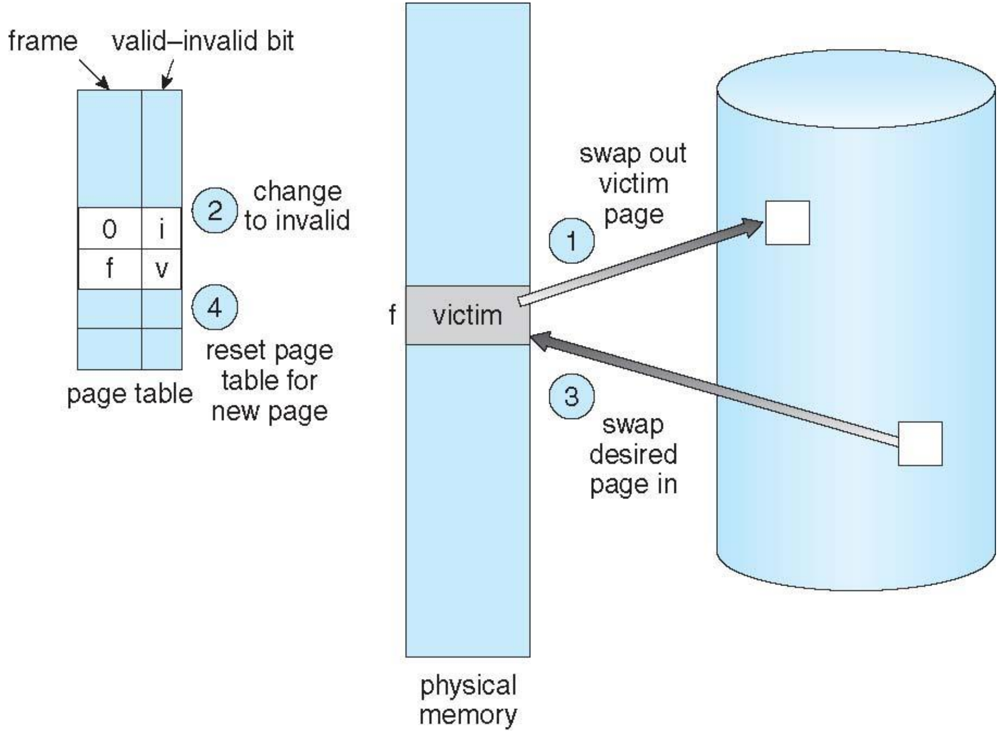
EAT = (1 – p) x memory access + p (page fault overhead

+ swap page out + swap page in)

Basic Page Replacement

- Find the location of the desired page on disk
- Find a free frame:
  - If there is a free frame, use it
  - If there is no free frame, use a page replacement algorithm to select a **victim frame**
  - Write victim frame to disk if dirty
- Bring the desired page into the (newly) free frame; update the page and frame tables
- Continue the process by restarting the instruction that caused the trap

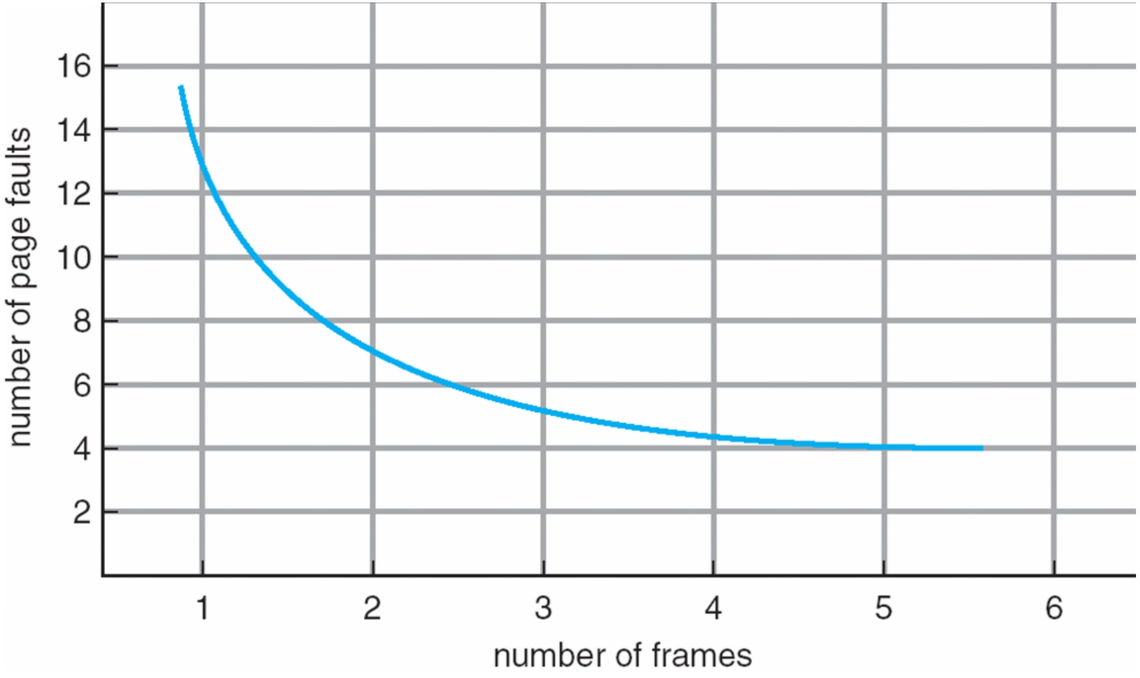
Page Replacement



Page Replacement Algorithm

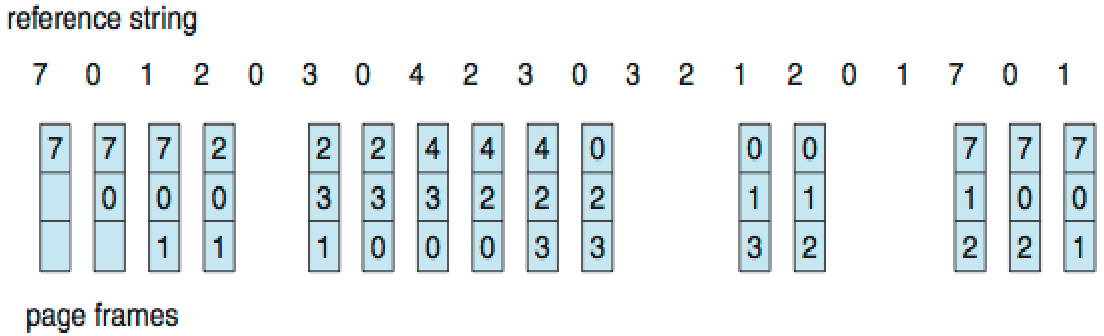
- **Page-replacement algorithm**
  - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
  - String is just page numbers, not full addresses
  - Repeated access to the same page does not cause a page fault
  - **FIFO, LIFO,**
  - **Optimal,**
  - **LRU, LRU approximations,**
  - **LFU, MFU**

Page Faults Vs Number of Frames



First-In-First-Out (FIFO) Algorithm

- Ref. string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

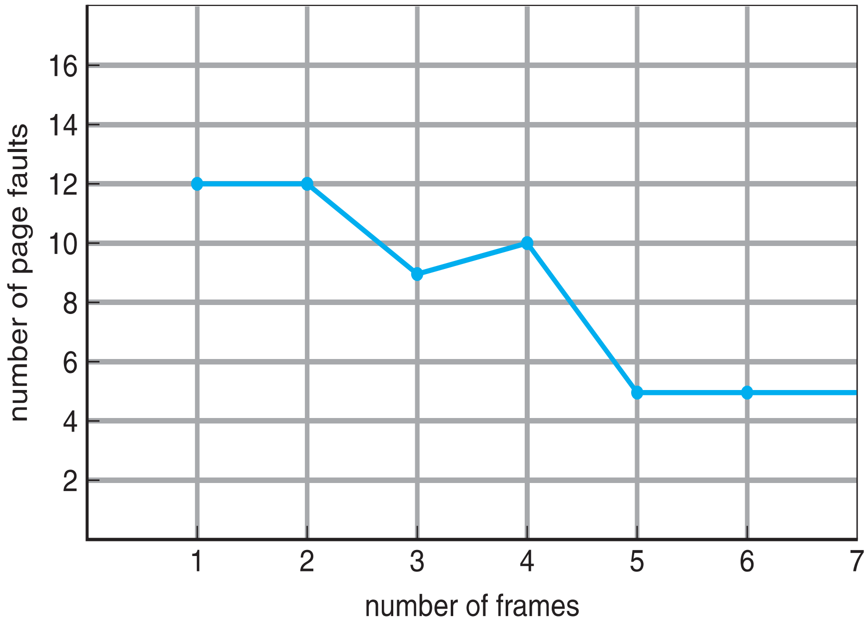


15 page faults

- How to track ages of pages? - Use a FIFO queue

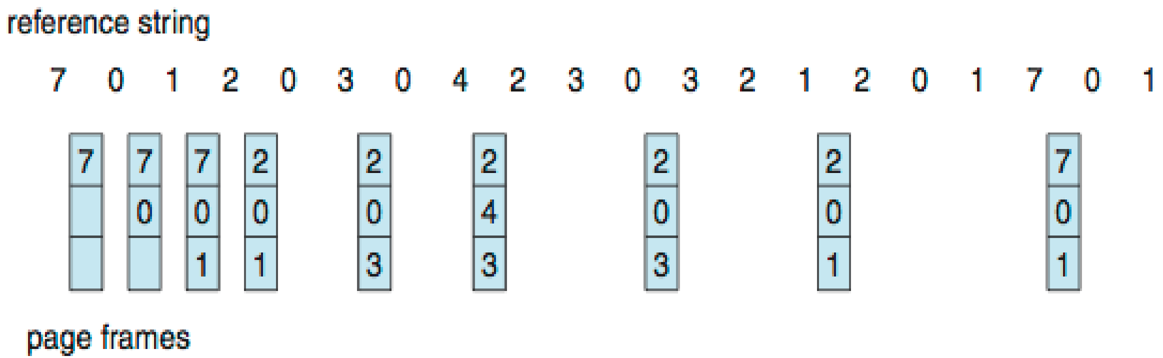
Belady's Anomaly

- Consider 1,2,3,4,1,2,5,1,2,3,4,5
- Adding more frames can cause more page faults! - **Belady's Anomaly**



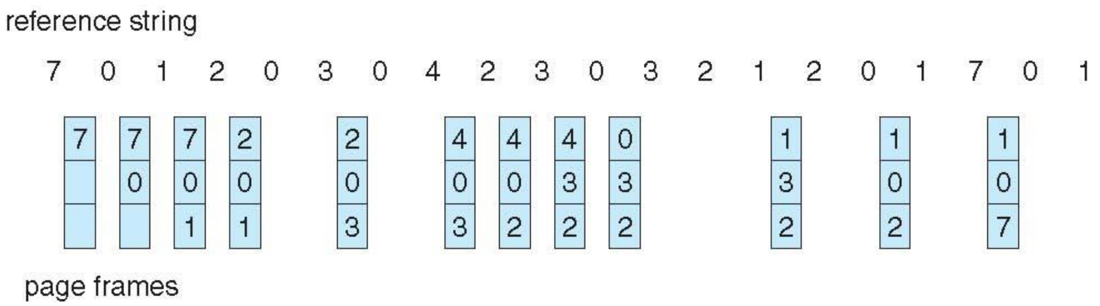
Optimal Algorithm

- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- Practical difficulty- Can’t read the future
- Used for measuring how well your algorithm performs



Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used

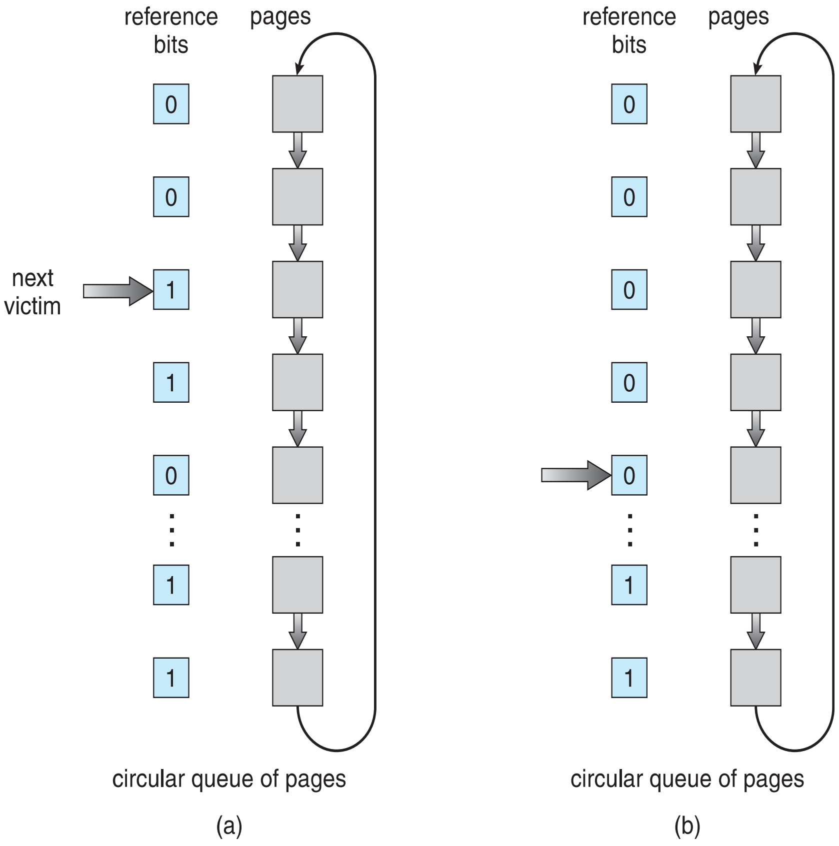
LRU Algorithm Implementation

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value.

LRU Approximation Algorithms

- **Reference bit**
  - With each page associate a bit, initially = 0
  - When page is referenced, bit set to 1
  - Replace any with reference bit = 0 (if one exists)
- **Second-chance algorithm**
  - If page to be replaced has
    - Reference bit = 0 →replace it
    - reference bit = 1 then:
      - set reference bit 0, leave page in memory
      - replace next page, subject to same rules

Second-Chance Page-Replacement



Enhanced Second-Chance Algorithm

- Improve algorithm by using reference bit and modify bit
- Take ordered pair (reference, modify)
- (0, 0) neither recently used not modified – best page to replace
- (0, 1) not recently used but modified – not quite as good, must write out before replacement
- (1, 0) recently used but clean – probably will be used again soon
- (1, 1) recently used and modified – probably will be used again soon and need to write out before replacement

Counting Algorithms

- Keep a counter of the number of references that have been made to each
- **Lease Frequently Used** (**LFU**) **Algorithm**: replaces page with smallest count
- **Most Frequently Used** (**MFU**) **Algorithm**: based on the argument that the page with the smallest count was probably just brought in and has yet to be used

Page-Buffering Algorithms

- Always keep a pool of free frames
  - When needed, frame is always available, not found at fault time
  - Read page into free frame and select victim to evict and add to free pool
  - When convenient, evict victim
- Keep list of modified pages
  - When free, write to backing store and set to non-dirty
- Possibly, keep free frame contents intact and note what is in them
  - If referenced again before reused, no need to load contents again from disk
  - Generally useful to reduce penalty if wrong victim frame selected

johnjose@iitg.ac.in
http://www.iitg.ac.in/johnjose/